

Lecture 4: Training Neural Networks

CS2140 Introduction to Machine Learning · Week 4 · Reading: Chapter 6

1. From Neurons to Networks

An **artificial neuron** is the basic unit of a neural network. It receives a vector of inputs x , multiplies each input by a **weight**, adds a **bias** term b , and passes the result through an **activation function**. The weighted sum $z = w \cdot x + b$ is called the pre-activation. The weights determine how strongly each input influences the output, and the bias shifts the decision boundary away from the origin.

The **perceptron**, introduced by Frank Rosenblatt in 1958, is the simplest neural network: a single neuron with a step activation. A perceptron can only separate data that is linearly separable. It famously cannot learn the XOR function, which motivated networks with more than one layer.

A **multilayer perceptron (MLP)** stacks neurons into layers: an input layer, one or more **hidden layers**, and an output layer. Each neuron in one layer connects to every neuron in the next layer (a fully connected or dense layer). Computing the output of the network from the inputs, layer by layer, is called the **forward pass** or forward propagation.

Key idea: without a non-linear activation function, any stack of layers collapses into a single linear transformation. Non-linearity is what gives deep networks their expressive power.

2. Activation Functions

The activation function introduces non-linearity. Three activation functions are used throughout this course:

- **Sigmoid**: $\sigma(z) = 1 / (1 + e^{-z})$ squashes any input into the range $(0, 1)$. It is useful for probabilities, but its gradient is at most 0.25 and approaches zero for large $|z|$, which causes the vanishing gradient problem in deep networks.
- **ReLU** (Rectified Linear Unit): $\text{ReLU}(z) = \max(0, z)$. ReLU is cheap to compute and its gradient is exactly 1 for positive inputs, so gradients flow well through many layers. Its weakness is the dying ReLU problem: a neuron whose input is always negative outputs zero and stops learning.
- **Softmax**: converts a vector of scores (logits) into a probability distribution that sums to 1. Softmax is used in the output layer for multi-class classification.

As a rule of thumb, use ReLU in hidden layers, sigmoid for a single binary output, and softmax for multi-class outputs.

3. Loss Functions

A **loss function** measures how far the predictions of the network are from the true targets on the training data. Training a network means finding the weights and biases that minimise the loss.

- **Mean Squared Error (MSE)** averages the squared difference between predictions and targets. It is the standard loss for regression tasks.
- **Cross-entropy loss** compares a predicted probability distribution with the true class. For a correct class with predicted probability p , the loss is $-\log(p)$, so confident wrong predictions are penalised heavily. Cross-entropy is paired with softmax (or sigmoid) outputs for classification.

The loss is a function of every weight in the network. We can picture it as a landscape over weight space; training searches this **loss landscape** for a low valley.

4. Gradient Descent

Gradient descent is the optimisation algorithm used to minimise the loss. The **gradient** of the loss with respect to the weights points in the direction of steepest increase, so we update each weight a small step in the opposite direction:

$$w \leftarrow w - \eta \cdot \partial L / \partial w \quad \text{where } \eta \text{ is the learning rate.}$$

The **learning rate** η is a hyperparameter that controls the step size. If it is too large, training diverges or oscillates around the minimum; if it is too small, training is very slow and may get stuck on plateaus. Typical starting values are 0.1 to 0.001.

Batch, stochastic and mini-batch

Batch gradient descent computes the gradient on the entire training set before each update, which is accurate but slow for large datasets. **Stochastic gradient descent (SGD)** updates the weights after every single example; updates are noisy but fast, and the noise can help escape shallow local minima. **Mini-batch gradient descent** uses small batches (for example 32 to 256 examples) and is the standard in practice because it balances stable gradients with efficient use of GPU hardware. One complete pass over the training set is called an **epoch**.

Momentum and adaptive optimisers such as **Adam** extend SGD. Momentum accumulates a running average of past gradients to smooth updates; Adam additionally adapts the learning rate of each weight individually.

5. Backpropagation

Gradient descent needs the gradient of the loss with respect to every weight. **Backpropagation** computes all of these gradients efficiently in a single backward pass through the network. It is an application of the **chain rule** from calculus: if the loss L

depends on an output a , which depends on a pre-activation z , which depends on a weight w , then $\partial L / \partial w = \partial L / \partial a \cdot \partial a / \partial z \cdot \partial z / \partial w$.

The network is viewed as a **computational graph**. During the forward pass each node stores its intermediate values. During the backward pass, starting from the loss, each node multiplies the incoming gradient by its local derivative and passes the result to the nodes that fed into it. Because intermediate gradients are reused, backpropagation costs roughly the same as one forward pass, regardless of the number of weights.

One training step therefore has four stages: (1) forward pass to compute predictions, (2) compute the loss, (3) backward pass (backpropagation) to compute gradients, and (4) a gradient descent update of the weights.

Vanishing and exploding gradients

Because backpropagation multiplies many local derivatives together, gradients can shrink exponentially as they travel back through many layers. This **vanishing gradient problem** means early layers learn extremely slowly; it is severe with sigmoid activations, whose derivative is at most 0.25. The opposite, **exploding gradients**, occurs when the factors are larger than 1 and causes unstable updates. ReLU activations, careful weight initialisation (such as He or Xavier initialisation), batch normalisation and gradient clipping all reduce these problems.

6. Generalisation, Overfitting and Regularisation

The goal of training is not a low training loss but good performance on unseen data, called **generalisation**. Data is therefore split into a **training set** used to fit the weights, a **validation set** used to choose hyperparameters and detect overfitting, and a **test set** used once at the end to estimate real-world performance.

Overfitting happens when a network memorises the training data, including its noise: the training loss keeps falling while the validation loss starts to rise. **Underfitting** is the opposite: the model is too simple, and both losses stay high. This trade-off is known as the bias-variance trade-off.

Regularisation refers to techniques that reduce overfitting:

- **L2 regularisation** (weight decay) adds a penalty proportional to the sum of squared weights to the loss, encouraging small weights and smoother functions.
- **Dropout** randomly sets a fraction of neurons (for example 50%) to zero during each training step, so the network cannot rely on any single neuron. Dropout is switched off at test time.
- **Early stopping** monitors the validation loss and stops training when it stops improving, keeping the best weights seen so far.
- **Data augmentation** enlarges the training set with transformed copies of examples, such as flipped or cropped images.

7. Summary and Checklist

To train a neural network: choose an architecture and activation functions; choose a loss that matches the task; initialise weights sensibly; then repeat forward pass, loss, backpropagation and a mini-batch gradient descent update for several epochs, while monitoring validation loss to tune the learning rate and apply regularisation.

Before next week: make sure you can (1) derive the gradient of a single sigmoid neuron with the chain rule, (2) explain why ReLU helps with vanishing gradients, and (3) read a training/validation loss curve and say whether a model is overfitting.